Лабораторна робота 6
Циклічні обчислювальні процеси. Функції.

Мета лабораторної роботи: набуття практичних навичок роботи з умовним оператором на мові C/C++ і розгалуженими алгоритмами; роботи з операторами циклу мови C/C++ і циклічними алгоритмами; організації наочного виведення інформації в текстовому режимі; програмної реалізації циклічних алгоритмів; налагодження програм, що містять циклічні алгоритми; складання циклічних алгоритмів із вкладеними циклами і розгалуженнями в тілі циклу.

I. Теми для попереднього опрацьовування

1. Логічні операції .
2. Умовний оператор мови C/C++.
3. Вкладений умовний оператор мови C/C++.
4. Оператор switch.
5. Тернарна умовна операція.
6. Оператори циклу мови C/C++.
7. Вкладені цикли.
8. Скорочена форма операції присвоювання: +=, -=, *=, /=, %=.
9. Постфіксний і суфіксний інкремент і декремент.
10. Функції. Поняття функції. Прототип функції.
11. Визначення та виклик функції.
12. Поняття формальних і фактичних параметрів.
13. Поняття локальних і глобальних змінних.
14. Передача параметрів функції та їх повернення.
15. Способи передачі параметрів функції: за значенням, за посиланням. Особливий випадок: передача вказівника за значенням.
16. Робота у режимі командного рядку.

II. Завдання

1. Складіть програму, яка виводить на екран таблицю значень функції F(x) у заданому діапазоні. Значення функції F(x) обчисліть за формулою, наведеною у вашому варіанті індивідуального завдання. Визначте області допустимих значень параметрів формул (у разі непопадання значення x до ОДЗ – видавати відповідне повідомлення). Діапазон і крок зміни аргументу користувач задає в командному рядку, в тому разі, якщо в командному рядку параметри не задано або їх задано в недостатній кількості, програма пропонує здійснити ввід з клавіатури під час виконання програми. Передбачити перевірку правильності введених користувачем аргументів. Значення F(x) обчислюють в окремій функції.

2. Складіть програму, яка обчислює значення функції, заданого за допомогою ряду Тейлора. Для ряду, члени якого обчислюються за формулою, що відповідає вашому індивідуальному завданню, підрахуйте та виведіть суму перших 5 членів ряду, суму членів ряду із уведеною точністю ε, кількість ітерацій та значення функції, що розрахована за допомогою стандартних математичних функцій. Організуйте вихід із програми за бажанням користувача.

Завдання виконайте за допомогою трьох операторів циклу.

III. Приклад розв’язання задачі (варіант 31)

1. Розробка алгоритму розв’язання другої задачі

Ряд, що розглядається як приклад, збігається для будь-якого дійсного x. Процес обчислення суми членів ряду має бути ітераційний – слід повторювати обчислення за однією і тією ж формулою у випадку таких значень лічильника: $i = 1, 2, ...$ У кожній ітерації циклу необхідно виконувати обчислення за заданою формулою для поточного значення члена ряду.

Одержане значення слід додавати до змінної, що являє собою суму ряду. Ця змінна в кожній ітерації міститиме суму всіх уже оброблених членів ряду. Її початкове значення має дорівнювати нулю або першому члену ряду.

Після обчислення кожного члена ряду (але до додавання його значення до суми) необхідно порівняти поточне значення із заданою межею точності. Через те, що значення члена ряду може бути як додатним, так і від’ємним, у ході порівняння слід використовувати абсолютне значення. Якщо абсолютне значення члена ряду стане меншим або дорівнюватиме ε, слід завершити обчислення.

Запишемо алгоритм обчислення суми в словесній формі:

1. Увести початкові дані (аргумент і точність).
2. Задати початкові значення номера члена ряду, першого члена ряду та суми ряду.
3. Організувати цикл:
   1) перевірити, чи досягнута точність обчислення;
   2) обчислити черговий член ряду;
   3) додати його до суми ряду.
4. Вивести значення функції.

Запишемо в загальному вигляді формулу для підрахунку n-го члена ряду:

$$C_n = (-1)^{n-1} \frac{x^{2n-1}}{(2n-1)!} \quad (n \text{ розпочинається із } 1).$$

На перший погляд може здатися, що необхідно організовувати цикли для обчислення факторіала і степенів. При цьому можна одержати дуже великі числа, у ході ділення яких одне на одне відбудеться втрата точності. Крім того, великі числа можуть переповнювати розрядну сітку.

Легко помітити, що $(n+1)$-й член ряду обчислюється після n-го, тому програма буде простіша та ефективніша, якщо обчислювати член ряду не з «нуля», а за допомогою множення попереднього члена на деяку величину. Знайдемо цю величину. Для цього запишемо формулу для $(n+1)$-го члена ряду, підставивши в попередній вираз $(n+1)$ замість n:

$$C_{n+1} = (-1)^n \frac{x^{2(n+1)-1}}{(2(n+1)-1)!}$$

Тепер знайдемо вираз, на який необхідно помножити C_n, щоб одержати C_{n+1}:

$$C_{n+1} = C_n \cdot \alpha \to \alpha = \frac{C_{n+1}}{C_n} = \frac{(-1)^n x^{2n+1}}{(2n+1)!} \cdot \frac{(2n-1)!}{(-1)^{n-1} x^{2n-1}} = -\frac{x^2}{2n(2n+1)} \Rightarrow$$

$$\Rightarrow C_{n+1} = -C_n \frac{x^2}{2n(2n+1)}.$$

2. Налагодження програми

Форма виведення в першій програмі така, що немає необхідності застосовувати в процесі налагодження покроковий режим. Складіть логічний план налагодження:

1) переконайтеся в правильній організації циклу;
2) переконайтеся в правильній роботі ланцюжка умов, які визначають відрізок;
3) переконайтеся в правильному обчисленні значення ординати на відрізках.

Якщо в ході обчислення виникають помилки складного виразу, можна включити в програму тимчасові додаткові змінні й розбити вираз на прості складники, перевіряючи крок за кроком значення цих змінних.

Під час налагодження другої програми можна застосувати покроковий режим із відстежуванням значень змінних. Слід виконати крок за кроком декілька ітерацій циклу, переконавшись, що змінні набувають правильних значень. У такому режимі неважко виконати 5-10 ітерацій і переконатися в правильному формуванні першого результату.

Для того щоб переконатися в правильності одержаного результату, доцільно у ході налагодження виводити разом із сумою і номер ітерації, на якому закінчилася робота циклу, і значення члена ряду, яке стало менше заданої точності.

До початку виконання програми неможливо передбачити, скільки членів ряду потрібно буде скласти. У циклі такого типу є небезпека, що він ніколи не завершиться через можливі помилки в обчисленнях або через обмежену область збіжності ряду. Тому для надійності програми необхідно передбачити аварійний вихід із циклу із виведенням попереджувального повідомлення після досягнення деякої максимально допустимої кількості ітерацій (наприклад, 2000 ітерацій).

Побудуйте програму так, щоб вона виводила не тільки значення аргументу і функції, але і кількість підсумованих членів ряду.

Виконайте програму кілька разів для різних значень аргументу і точності.

Знайдіть залежність між цими величинами.

3. Текст програми

```
/*********************************************************/
/*                Лабораторна робота 6.1                 */
/*         Приклад обчислення значення функції            */
/*                  Приклад виконання.                    */
/*                     Варіант 31                         */
/*********************************************************/
#include <iostream>
#include <math.h>
#include<Windows.h>
int main(int argc, char *argv[]) {

double x, xn, xk, shag, y;
if (argc == 4) {
     xn = atof(argv[1]);
     xk = atof(argv[2]);
     shag = atof(argv[3]);
}
else {
     cout << "Vvedite xn, xk, shag \n";
     cin >> xn >> xk >> shag;
}

cout << "|-----------|-----------------|\n";
cout << "|     x     |         y       |\n";
cout << "|-----------|-----------------|\n";

for (x = xn; x <= xk; x += shag) {
     if (x <= 0) y = pow(x, 3);
     else if (x > 10) y = sqrt(x);
     else if (x < 9) y= 5.7 - 2 * sqrt(x);
     else continue;
     cout<<"|   "<< setw(8)<<x<< "| " << setw(15) << y << "
|\n";
}
cout << "|-----------|-----------------|\n";
return 0;
}
```

Результат роботи програми

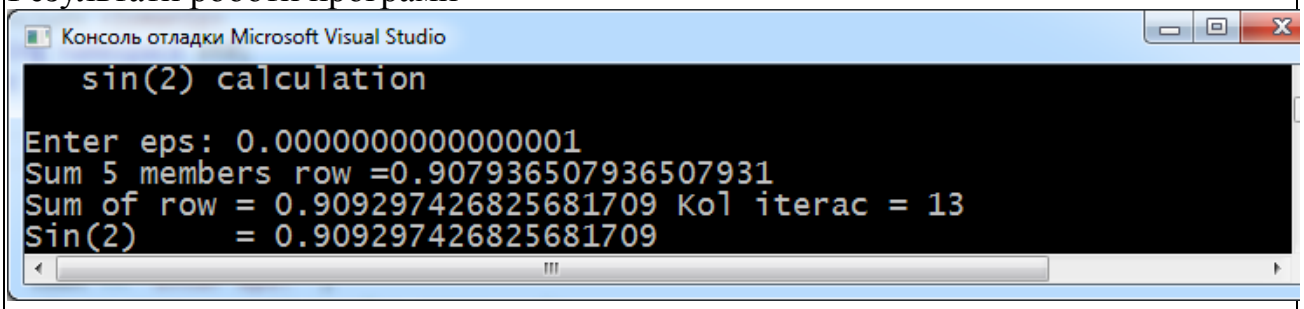
Консоль отладки Microsoft Visual Studio

```
Vvedite xn, xk, shag
-2 15 1.5
|-----------|----------------|
|     x     |        y       |
|-----------|----------------|
|         -2|              -8 |
|       -0.5|          -0.125 |
|          1|             3.7 |
|        2.5|         2.53772 |
|          4|             1.7 |
|        5.5|         1.00958 |
|          7|        0.408497 |
|        8.5|       -0.130952 |
|       11.5|         3.39116 |
|         13|         3.60555 |
|       14.5|         3.80789 |
|-----------|----------------|

C:\Users\User\source\repos\ConsoleApplication160\Debug\Consc
```

```
/*****************************************************/
/*            Лабораторна робота 6.2                 */
/*             Обчислення суми ряду                  */
/*          Приклад виконання. Варіант 31            */
/*****************************************************/
#include <iostream>
#include <math.h>
#include<Windows.h>
#include <iomanip>
using namespace std;
int main() {
 double i;
 double x, sum;// аргумент функції, сума ряду
 double item;    // поточне значення  члена ряду
 double eps;     // межа точності
 cout<<"   sin(2) calculation\n\n";
 cout<<"Enter eps: ";
 cin>> eps;      // похибка розрахунків
 x = 2;               // значення аргументу
 sum = x;          // початкове значення суми ряду
 item = x;          // перший член ряду
 for (i = 2; fabs(item) > eps; i+=2, sum +=item)  {
   item *=(-x*x)/(i*(i+1));// черговий член ряду
   if (i==8)       // 5 членів - виведення суми
   cout << "Sum 5 members row =" << setprecision(18) <<
sum << endl;
    } //  end  for
 // виведення остаточної суми
 cout << setprecision (18) << "Sum of row = "<< sum << "
Kol iterac = "<<i/2<<"\n";
 cout<< "Sin(2)     = " << sin(x) <<endl;
 return 0;
}
```

Результати роботи програми



IV. Довідковий матеріал

IV.1 Розглянемо деякі стандартні математичні функції (таблиця).

Стандартні математичні функції знаходяться у файлі <math.h> (<cmath>).

Ім'я	Дія	Прототип
abs	Абсолютне значення цілого числа - |x|	int abs(int x);
labs	Абсолютне значення "довгого" цілого числа	long labs(long x);
fabs	Абсолютне значення числа з плаваючою точкою	double fabs(double x);
sqrt	Добування квадратного кореня	double sqrt(double x);
pow	Піднесення до степеня	double pow(double x, double y);
cos	Обчислення косинуса[1]	double cos(double x);
sin	Обчислення синуса аргументу	double sin(double x);
tan	Обчислення тангенса аргументу	double tan(double x);
acos	Обчислення арккосинуса аргументу	double acos(double x);
asin	Обчислення арксинуса аргументу	double asin(double x);
atan	Обчислення арктангенса аргументу	double atan(double x);
atan2	Обчислення арктангенса відношення аргументів	double atan2(double x, double y);
exp	Обчислення степеня числа *e*	double exp(double x);
log	Обчислення натурального логарифма	double log(double x);
log10	Обчислення десяткового логарифма	double log10(double x);

Проаналізуємо деякі функції стандартного введення-виведення.

IV.2 Мова C/C++ має механізм, що дозволяє програмі отримати доступ до аргументів командного рядку. Для цього використовуємо наступний заголовок функції main():

int main(int argc, char *argv[])

Аргумент argc визначає кількість аргументів командного рядку, включаючи і саме ім'я програми. Змінна argv – це покажчик на тип char. Змінну argv можна трактувати як масив покажчиків на аргументи командного рядку, причому argv[0] – це покажчик на перший символ рядку, що містить ім'я програми, argv[1] – це покажчик на перший символ рядку, що містить перший аргумент командного рядку тощо.

Наприклад, командний рядок має вигляд:

krok.exe temp1 temp2 temp3

Тоді argc дорівнює 4, argv[0] дорівнює krok, argv[1] дорівнює temp1, argv[2] дорівнює temp2, argv[3] дорівнює temp3.

IV.3 Степеневі ряди – це один із інструментів обчислення математичних функцій **sin*x***, **cos*x***, $\mathbf{e^x}$, **ln*x***, **arctg*x*** тощо. Наближене значення суми ряду можна одержати або обмежуючись сумою перших *n* його членів, або обчислюючи суму із наперед заданою точністю. Із заданою точністю може бути обчислена сума

[1] Тут і далі аргумент задається в радіанах.

лише збіжного ряду, а довільний степеневий ряд має певну область збіжності (можливо, порожню), тобто збігається не за всіх, а лише за деяких значень *x*. Визначитись, скільки членів ряду необхідно скласти для досягнення точності, неможливо, оскільки під час аналізу умови виходу застосовується змінна, яка підраховується на кожній ітерації циклу. Такі цикли небезпечні, тому для попередження зациклювання необхідно передбачити аварійний вихід із циклу в разі досягнення деякої максимально допустимої кількості ітерацій із видачою попереджувального повідомлення.

Інший інструмент – це ланцюгові, або неперервні, дроби. Обчислення математичних функцій **sin*x***, **cos*x***, $\mathbf{e^x}$, **ln*x***, **arctg*x*** у вбудованих функціях реалізується за допомогою ланцюгових дробів. Їх обчислюють за допомогою рекурентного співвідношення.

Формула, що виражає член числової послідовності через один або декілька попередніх членів, називається рекурентним співвідношенням. Послідовність, члени якої задовольняють певне рекурентне співвідношення, називається рекурентною.

Розглянемо задачу наближеного обчислення значення суми: **S = 1/1! + 1/2! + ... + 1/k!**

Можна, звичайно, кожного разу обчислювати чергове **p!**, потім **1/p!** і одержаний доданок додавати до суми. Але звернемо увагу на очевидну рівність **1/(p+1)! = (1/p!)/(p+1)**. Програма обчислення запишеться таким чином:

```
double S, p;
int k = 18;
S = p = 1;//S = p = 1/1!
for (int i = 2; i <= k; ++i) {
        p = p/i;
        S += p;
}
cout<<S<<endl;
```

Контрольні запитання й завдання

1. Обчисліть із заданою точністю константу $\pi \approx 3.14159265$ за допомогою ряду Лейбніца $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots$
2. У чому полягає технологія низхідного проектування програм?
3. У чому відмінність формальних параметрів від фактичних?
4. Як повертаються значення із функції? Скільки значень повертає функція?
5. Напишіть функцію для обчислення довжини сторін трикутника, заданого координатами вершин, а також функцію для обчислення площі трикутника за формулою Герона.
6. Командний рядок має вигляд:
   trc.exe test1 test2 test3
   У наступному циклі кожний елемент командного рядку виводиться в окремому рядку:
   for (int i=1; i<argc; i++) cout<<argv[i]<<endl;
   Що буде виведено на екран? Що зміниться, якщо замість int i=1 записати int i=0?